



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

Notebook 02-A: Verificar la Instalación de Postgres

Objetivos:

1. Verificación de Postgres
2. Verificación de MinIO con boto3
3. Escritura de Delta Lake en bucket S3 (MinIO)
4. Lectura y transformación desde MinIO
5. Pipeline end-to-end: MinIO → Spark → PostgreSQL

Artefactos creados:

- Bucket: spark-lake
- Delta tables: transacciones, transacciones_transformadas
- PostgreSQL tables: transacciones_procesadas

Preliminaries:

Asegúrese de estar en directorio project y activar ambiente de Python

```
Shell

# Activiar ambiente de Python
source ~/project/pgvd_env/bin/activate

# Entrar a la carpeta project
cd ~/project

# Inicie Notebook de Jupyter
./start_jupyter.sh
```

Celda 1: Configuración de Conexión JDBC

```
Python

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, lit, current_date, sum as spark_sum,
count, avg
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
# Configuración de conexión PostgreSQL
jdbc_url = "jdbc:postgresql://localhost:5432/db_sports"

connection_properties = {
    "user": "postgres",
    "password": "123abc",
    "driver": "org.postgresql.Driver"
}

print("Configuración JDBC definida:")
print(f"URL: {jdbc_url}")
print(f"Driver: {connection_properties['driver']}")
```

Salida esperada:

```
Shell
Configuración JDBC definida:
URL: jdbc:postgresql://localhost:5432/db_sports
Driver: org.postgresql.Driver
```

Celda 2: Crear SparkSession con soporte JDBC

```
Python
spark = (SparkSession.builder
    .appName("ETL_PostgreSQL")
    .master("local[*]")
    .config("spark.driver.memory", "1g")
    .config("spark.executor.memory", "1500m")
    .config("spark.jars", "/usr/local/spark-3.5.8/jars/postgresql-42.7.3.jar")
    .getOrCreate())

print("SparkSession creada con soporte JDBC")
print(f"App: {spark.sparkContext.appName}")
print(f"Version: {spark.version}")
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Salida esperada:

```
Shell
SparkSession creada con soporte JDBC
App: ETL_PostgreSQL
Version: 3.5.8
```

Celda 3: Leer Datos desde PostgreSQL

```
Python
print("=" * 60)
print("LECTURA DESDE POSTGRESQL")
print("=" * 60)

# Leer tabla affiliations
df_affiliations = spark.read.jdbc(
    url=jdbc_url,
    table="affiliations",
    properties=connection_properties
)

print("Tabla 'addresses':")
df_affiliations.printSchema()
df_affiliations.show()
```

Salida esperada:

```
Shell
=====
LECTURA DESDE POSTGRESQL
=====
Tabla 'affiliations':

root
 |-- id: integer (nullable = true)
 |-- affiliation_key: string (nullable = true)
 |-- affiliation_type: string (nullable = true)
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
|-- publisher_id: integer (nullable = true)
```

+---+-----+-----+-----+			
id	affiliation_key	affiliation_type	publisher_id
+---+-----+-----+-----+			
1	l.mlb.com	league	1
2	15007000	sport	1
3	l.nfl.com	league	1
4	15003000	sport	1
5	c.afc	conference	1
6	d.afceast	division	1
7	d.afcnorth	division	1
8	d.afcsouth	division	1
9	d.afcwest	division	1
10	c.nfc	conference	1
11	d.nfceast	division	1
12	d.nfcnorth	division	1
13	d.nfcsouth	division	1
14	d.nfcwest	division	1
15	l.nba.com	league	1
16	15008000	sport	1
17	c.eastern	conference	1
18	d.atlantic	division	1
19	d.central	division	1
20	d.southeast	division	1
+---+-----+-----+-----+			



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

Notebook 02-B: Verificar la Configuración MinIO

Objetivo:

Integrar Spark con MinIO (S3 local) para operaciones de Data Lake: lectura/escritura de Delta Lake en buckets S3.

Celda 1: Verificar MinIO activo

```
Python
import subprocess
import os

print("=" * 60)
print("VERIFICACIÓN DE MINIO")
print("=" * 60)

# Verificar que MinIO está corriendo
result = subprocess.run(["pgrep", "-f", "minio"], capture_output=True, text=True)
if result.returncode == 0:
    print("Proceso MinIO encontrado")
else:
    print("MinIO no está corriendo. Ejecute ~/minio/start_minio.sh")

# Verificar puertos
for port in [9000, 9001]:
    result = subprocess.run(["nc", "-z", "localhost", str(port)],
capture_output=True)
    status = "Abierto" if result.returncode == 0 else "Cerrado"
    print(f" Puerto {port}: {status}")

print("\nConfiguración S3:")
print("  Endpoint: http://localhost:9000")
print("  Access Key: minioadmin")
print("  Secret Key: minioadmin123")
print("  Bucket: spark-lake")
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Salida esperada:

```
Shell
=====
VERIFICACIÓN DE MINIO
=====
Proceso MinIO encontrado
Puerto 9000: Abierto
Puerto 9001: Abierto

Configuración S3:
Endpoint: http://localhost:9000
Access Key: minioadmin
Secret Key: minioadmin123
Bucket: spark-lake
```

Celda 2: Verificar MinIO activo

```
Python
from pyspark.sql import SparkSession

print("=" * 60)
print("CREACIÓN DE SPARKSESSION CON SOPORTE S3A")
print("=" * 60)

spark = (SparkSession.builder
    .appName("Spark & Data Lake")
    .master("local[*]")
    .config("spark.driver.memory", "1g")
    .config("spark.executor.memory", "1500m")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.hadoop.fs.s3a.endpoint", "http://localhost:9000")
    .config("spark.hadoop.fs.s3a.access.key", "minioadmin")
    .config("spark.hadoop.fs.s3a.secret.key", "minioadmin123")
    .config("spark.hadoop.fs.s3a.path.style.access", "true")
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

```
.config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
.config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
.config("spark.hadoop.fs.s3a.aws.credentials.provider",
"org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider")
.config("spark.hadoop.fs.s3a.endpoint.region", "us-east-1")
.getOrCreate()

print("SparkSession creada exitosamente")
print(f"App: {spark.sparkContext.appName}")
print(f"Version: {spark.version}")
```

Salida esperada:

Shell

```
=====
CREACIÓN DE SPARKSESSION CON SOPORTE S3A
=====
SparkSession creada exitosamente
App: Spark & Data Lake
Version: 3.5.8
```

Celda 3: Verificar Bucket con Boto3

Python

```
import boto3
from botocore.exceptions import ClientError

print("=" * 60)
print("VERIFICACIÓN DE BUCKET CON BOTO3")
print("=" * 60)

# Cliente S3 apuntando a MinIO
s3 = boto3.client(
    's3',
    endpoint_url='http://localhost:9000',
```

```
aws_access_key_id='minioadmin',
aws_secret_access_key='minioadmin123',
region_name='us-east-1'
)

# Listar buckets
try:
    response = s3.list_buckets()
    print("Buckets disponibles:")
    for bucket in response['Buckets']:
        print(f"    - {bucket['Name']} (creado: {bucket['CreationDate']})")
except ClientError as e:
    print(f" Error: {e}")

# Crear bucket si no existe
bucket_name = "spark-lake"
try:
    s3.head_bucket(Bucket=bucket_name)
    print(f"\nBucket '{bucket_name}' ya existe")
except ClientError:
    s3.create_bucket(Bucket=bucket_name)
    print(f"\nBucket '{bucket_name}' creado")
```

Salida esperada:

```
Shell
=====
VERIFICACIÓN DE BUCKET CON BOT03
=====
Buckets disponibles:
=spark-lake (creado: 2026-09-17 22:46:43.513000+00:00)
Bucket 'spark-lake' ya existe
```

Celda 4: Crear Datos de Prueba y Escribir a MinIO (Delta)

Python

```
from pyspark.sql.functions import col, lit, current_date, rand, when
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType, DateType
```

```
print("=" * 60)
print("ESCRITURA DE DELTA LAKE A MINIO")
print("=" * 60)
```

Crear DataFrame de transacciones

```
data = [
    ("TXN001", "C001", "Laptop", 1, 2500.00, "2024-01-15"),
    ("TXN002", "C002", "Mouse", 3, 25.50, "2024-01-16"),
    ("TXN003", "C001", "Teclado", 1, 120.00, "2024-01-17"),
    ("TXN004", "C003", "Monitor", 2, 450.00, "2024-01-18"),
    ("TXN005", "C002", "Webcam", 1, 80.00, "2024-01-19"),
]
```

```
schema = StructType([
    StructField("id_transaccion", StringType(), False),
    StructField("id_cliente", StringType(), False),
    StructField("producto", StringType(), False),
    StructField("cantidad", IntegerType(), False),
    StructField("precio", DoubleType(), False),
    StructField("fecha", StringType(), False)
])
```

```
df_transacciones = spark.createDataFrame(data, schema)
df_transacciones = df_transacciones.withColumn("fecha",
col("fecha").cast(DateType()))
```

```
print("DataFrame de transacciones:")
df_transacciones.show()
```

Ruta S3A

```
delta_path = "s3a://spark-lake/transacciones"
```

```
print(f"\nEscribiendo en: {delta_path}")
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

```
df_transacciones.write.format("delta").mode("overwrite").save(delta_path)
print("Datos escritos en Delta Lake sobre MinIO")
```

Salida esperada:

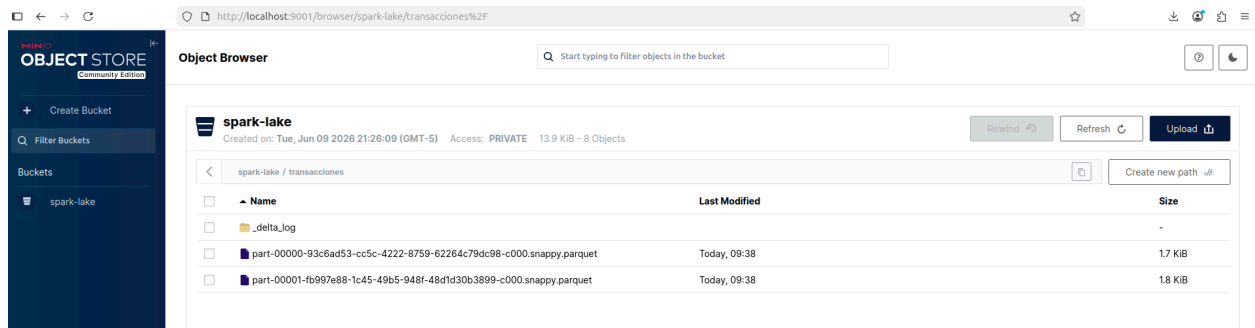
Shell

```
=====
ESCRITURA DE DELTA LAKE A MINIO
=====
```

DataFrame de transacciones:

id_transaccion	id_cliente	producto	cantidad	precio	fecha
TXN001	C001	Laptop	1	2500.0	2024-01-15
TXN002	C002	Mouse	3	25.5	2024-01-16
TXN003	C001	Tecclado	1	120.0	2024-01-17
TXN004	C003	Monitor	2	450.0	2024-01-18
TXN005	C002	Webcam	1	80.0	2024-01-19

Escribiendo en: s3a://spark-lake/transacciones
Datos escritos en Delta Lake sobre MinIO





PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

Celda 5: Leer desde MinIO y Transformar

Python

```
print("=" * 60)
print("LECTURA Y TRANSFORMACIÓN DESDE MINIO")
print("=" * 60)

# Leer desde Delta en MinIO
df_delta = spark.read.format("delta").load(delta_path)
print("Datos leídos desde Delta Lake/MinIO:")
df_delta.show()

# Transformaciones
df_transformado = df_delta.withColumn(
    "total_linea", col("cantidad") * col("precio")
).withColumn(
    "categoria",
    when(col("total_linea") > 1000, "Alto Valor")
    .when(col("total_linea") > 100, "Medio Valor")
    .otherwise("Bajo Valor")
).withColumn("fecha_procesamiento", current_date())

print("\nDataFrame transformado:")
df_transformado.show()

# Escribir resultado transformado
delta_transformado = "s3a://spark-lake/transacciones_transformadas"
df_transformado.write.format("delta").mode("overwrite").save(delta_transformado)
print(f"\nDatos transformados escritos en: {delta_transformado}")
```

Salida esperada:

Shell

```
=====
LECTURA Y TRANSFORMACIÓN DESDE MINIO
=====
Datos leídos desde Delta Lake/MinIO:
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

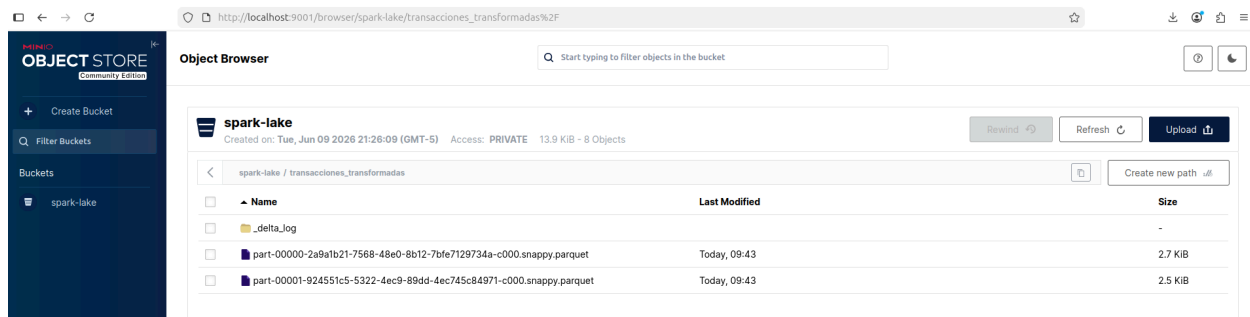
Departamento de Electrónica y Ciencias de la Computación

```
+-----+-----+-----+-----+-----+-----+
|id_transaccion|id_cliente|producto|cantidad|precio|      fecha|
+-----+-----+-----+-----+-----+-----+
|      TXN003 |      C001 | Teclado |      1 | 120.0 | 2024-01-17 |
|      TXN004 |      C003 | Monitor |      2 | 450.0 | 2024-01-18 |
|      TXN005 |      C002 | Webcam |      1 |  80.0 | 2024-01-19 |
|      TXN001 |      C001 | Laptop |      1 |2500.0 | 2024-01-15 |
|      TXN002 |      C002 | Mouse  |      3 |  25.5 | 2024-01-16 |
+-----+-----+-----+-----+-----+-----+
```

DataFrame transformado:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|id_transaccion|id_cliente|producto|cantidad|precio|      fecha|total_linea|  categoria|fecha_procesamiento|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      TXN003 |      C001 | Teclado |      1 | 120.0 | 2024-01-17 |    120.0 | Medio Valor |    2026-06-10 |
|      TXN004 |      C003 | Monitor |      2 | 450.0 | 2024-01-18 |    900.0 | Medio Valor |    2026-06-10 |
|      TXN005 |      C002 | Webcam |      1 |  80.0 | 2024-01-19 |     80.0 | Bajo Valor |    2026-06-10 |
|      TXN001 |      C001 | Laptop |      1 |2500.0 | 2024-01-15 |   2500.0 | Alto Valor |    2026-06-10 |
|      TXN002 |      C002 | Mouse  |      3 |  25.5 | 2024-01-16 |     76.5 | Bajo Valor |    2026-06-10 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Datos transformados escritos en: s3a://spark-lake/transacciones_transformadas





PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Notebook 02-C: Pipeline Completo: MinIO --> Spark → PostgreSQL

Python

```
from pyspark.sql import SparkSession

print("=" * 60)
print("CREACIÓN DE SPARKSESSION CON SOPORTE S3A")
print("=" * 60)

spark = (SparkSession.builder
    .appName("Pipeline Completo")
    .master("local[*]")
    .config("spark.driver.memory", "1g")
    .config("spark.executor.memory", "1500m")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.hadoop.fs.s3a.endpoint", "http://localhost:9000")
    .config("spark.hadoop.fs.s3a.access.key", "minioadmin")
    .config("spark.hadoop.fs.s3a.secret.key", "minioadmin123")
    .config("spark.hadoop.fs.s3a.path.style.access", "true")
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
    .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
"org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider")
    .config("spark.hadoop.fs.s3a.endpoint.region", "us-east-1")
    .getOrCreate())

print("SparkSession creada exitosamente")
print(f"App: {spark.sparkContext.appName}")
print(f"Version: {spark.version}")
```

Python

```
print("=" * 60)
print("PIPELINE END-TO-END: MINIO → SPARK → POSTGRESQL")
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

```
print("=" * 60)

# Leer datos transformados desde MinIO
delta_transformado = "s3//spark-lake/transacciones_transformadas"
df_final = spark.read.format("delta").load(delta_transformado)

# Configuración JDBC (misma del Notebook 03)
jdbc_url = "jdbc:postgresql://localhost:5432/db_sports"
connection_properties = {
    "user": "postgres",
    "password": "abc123",
    "driver": "org.postgresql.Driver"
}

# Escribir a PostgreSQL
df_final.write.jdbc(
    url=jdbc_url,
    table="transacciones_procesadas",
    mode="overwrite",
    properties=connection_properties
)

print("Datos de MinIO escritos en PostgreSQL (tabla: transacciones_procesadas)")

# Verificar lectura desde PostgreSQL
df_verificacion = spark.read.jdbc(
    url=jdbc_url,
    table="transacciones_procesadas",
    properties=connection_properties
)

print("\nVerificación - Datos en PostgreSQL:")
df_verificacion.show()
# Conteos cruzados
count_minio = df_final.count()
count_pg = df_verificacion.count()
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

```
print(f"\nConteo en MinIO: {count_minio}")
print(f"Conteo en PostgreSQL: {count_pg}")
print(f"Integridad verificada: {'CORRECTA' if count_minio == count_pg else 'ERROR'}")
```

Salida esperada:

Shell

```
=====
PIPELINE END-TO-END: MINIO → SPARK → POSTGRESQL
=====
```

Datos de MinIO escritos en PostgreSQL (tabla: transacciones_procesadas)

Verificación - Datos en PostgreSQL:

id_transaccion	id_cliente	producto	cantidad	precio	fecha	total_linea	categoria	fecha_procesamiento
TXN001	C001	Laptop	1	2500.0	2024-01-15	2500.0	Alto Valor	2026-06-10
TXN002	C002	Mouse	3	25.5	2024-01-16	76.5	Bajo Valor	2026-06-10
TXN003	C001	Teclado	1	120.0	2024-01-17	120.0	Medio Valor	2026-06-10
TXN004	C003	Monitor	2	450.0	2024-01-18	900.0	Medio Valor	2026-06-10
TXN005	C002	Webcam	1	80.0	2024-01-19	80.0	Bajo Valor	2026-06-10

Conteo en MinIO: 5

Conteo en PostgreSQL: 5

Integridad verificada: CORRECTA

Detener Spark

Python

```
spark.stop()
print("\n NOTEBOOK 04 COMPLETADO")
```